

**2026Summer**

**环境--作物--管理互作模型**

**技术培训手册：TRY&GET STARTED**

彭宇程 | 2026 年 7 月 | 版本 1.0

## 目录

|                               |    |
|-------------------------------|----|
| 第一部分 项目总览与协作基础 .....          | 7  |
| 第 1 章 项目架构与分工设计 .....         | 7  |
| 1.1 你能学到什么 .....              | 7  |
| 1.2 项目背景 .....                | 7  |
| 1.3 总体技术架构 .....              | 8  |
| 1.4 团队分工设计 .....              | 8  |
| 1.5 模块间接口约定 .....             | 9  |
| 1.6 掌握后的延展能力 .....            | 10 |
| 第 2 章 Git 与 GitHub 团队协作 ..... | 11 |
| 2.1 你能学到什么 .....              | 11 |
| 2.2 为什么 Git 是必备技能 .....       | 11 |
| 2.3 项目仓库结构 .....              | 11 |
| 2.4 分支策略 .....                | 12 |
| 2.5 日常工作流 .....               | 13 |
| 2.6 提交信息规范 .....              | 13 |
| 2.7 掌握后的延展能力 .....            | 14 |
| 第 3 章 Python 开发环境与项目管理 .....  | 15 |
| 3.1 你能学到什么 .....              | 15 |
| 3.2 环境隔离的重要性 .....            | 15 |
| 3.3 推荐环境方案 .....              | 15 |
| 3.4 项目文件组织规范 .....            | 16 |
| 3.5 掌握后的延展能力 .....            | 17 |
| 第二部分 数据获取与预处理 .....           | 19 |

|                                  |    |
|----------------------------------|----|
| 第 4 章 地理空间数据处理 .....             | 19 |
| 4.1 你能学到什么 .....                 | 19 |
| 4.2 栅格数据的本质 .....                | 19 |
| 4.3 核心操作 .....                   | 19 |
| 4.4 与项目管理衔接 .....                | 20 |
| 4.5 常见问题与优化 .....                | 21 |
| 4.6 掌握后的延展能力 .....               | 22 |
| 第 5 章 多源数据融合与特征工程 .....          | 23 |
| 5.1 你能学到什么 .....                 | 23 |
| 5.2 为什么特征工程决定模型上限 .....          | 23 |
| 5.3 数据融合流程 .....                 | 23 |
| 5.4 核心代码示例 .....                 | 25 |
| 5.5 掌握后的延展能力 .....               | 26 |
| 第 6 章 环境指纹构建 .....               | 27 |
| 6.1 你能学到什么 .....                 | 27 |
| 6.2 什么是"环境指纹" .....              | 27 |
| 6.3 结构程序设计 .....                 | 27 |
| 6.4 质量控制 .....                   | 29 |
| 6.5 掌握后的延展能力 .....               | 29 |
| 第三部分 建模与优化 .....                 | 31 |
| 第 7 章 机器学习建模：从决策树到 XGBoost ..... | 31 |
| 7.1 你能学到什么 .....                 | 31 |
| 7.2 从决策树到梯度提升：完整推导 .....         | 31 |
| 7.3 本项目中的建模设计 .....              | 33 |

|                                  |    |
|----------------------------------|----|
| 7.4 核心代码 .....                   | 33 |
| 7.5 训练技巧与常见陷阱 .....              | 35 |
| 7.6 掌握后的延展能力 .....               | 35 |
| 第 8 章 模型解释：SHAP 值原理与应用 .....     | 37 |
| 8.1 你能学到什么 .....                 | 37 |
| 8.2 Shapley 值的直觉理解 .....         | 37 |
| 8.3 核心代码 .....                   | 38 |
| 8.4 SHAP 在育种中的应用 .....           | 38 |
| 8.5 掌握后的延展能力 .....               | 39 |
| 第 9 章 多目标优化：NSGA-II .....        | 40 |
| 9.1 你能学到什么 .....                 | 40 |
| 9.2 为什么多目标优化在农业中如此重要 .....       | 40 |
| 9.3 NSGA-II 算法原理 .....           | 40 |
| 9.4 核心代码 .....                   | 41 |
| 9.5 NSGA-II 的局限性及本项目中的应对 .....   | 43 |
| 9.6 掌握后的延展能力 .....               | 43 |
| 第 10 章 不确定性量化：Bootstrap 方法 ..... | 45 |
| 10.1 你能学到什么 .....                | 45 |
| 10.2 为什么不确定性量化至关重要 .....         | 45 |
| 10.3 Bootstrap 原理 .....          | 45 |
| 10.4 核心代码 .....                  | 46 |
| 10.5 不确定性分解 .....                | 47 |
| 10.6 掌握后的延展能力 .....              | 47 |
| 第四部分 R 语言统计与可视化 .....            | 49 |

|                            |    |
|----------------------------|----|
| 第 11 章 R 语言统计分析.....       | 49 |
| 11.1 你能学到什么 .....          | 49 |
| 11.2 为什么项目需要 R .....       | 49 |
| 11.3 R 与 Python 的互操作 ..... | 50 |
| 11.4 核心操作示例.....           | 50 |
| 11.5 掌握后的延展能力 .....        | 51 |
| 第 12 章 R 语言科学可视化.....      | 52 |
| 12.1 你能学到什么 .....          | 52 |
| 12.2 ggplot2 语法哲学.....     | 52 |
| 12.3 核心代码示例.....           | 52 |
| 12.4 图表规范.....             | 54 |
| 12.5 掌握后的延展能力 .....        | 54 |
| 第五部分 部署与可重复性.....          | 55 |
| 第 13 章 模型部署与 API 服务.....   | 55 |
| 13.1 你能学到什么 .....          | 55 |
| 13.2 为什么需要部署 .....         | 55 |
| 13.3 核心代码.....             | 55 |
| 13.4 掌握后的延展能力 .....        | 57 |
| 第 14 章 容器化与可重复研究 .....     | 58 |
| 14.1 你能学到什么 .....          | 58 |
| 14.2 "在我的机器上能跑"的问题.....    | 58 |
| 14.3 项目 Dockerfile.....    | 58 |
| 14.4 可重复研究的最佳实践.....       | 59 |
| 14.5 掌握后的延展能力 .....        | 60 |

|                            |    |
|----------------------------|----|
| 第 15 章 学术写作与文献管理 .....     | 61 |
| 15.1 你能学到什么 .....          | 61 |
| 15.2 文献管理推荐：Zotero .....   | 61 |
| 15.3 论文结构设计（如需撰写） .....    | 62 |
| 15.4 写作规范与去 AI 化 .....     | 62 |
| 15.5 掌握后的延展能力 .....        | 63 |
| 附录 A 推荐学习路径（3 周速成方案） ..... | 64 |
| 附录 B 常见问题排查 .....          | 64 |
| 附录 C 术语对照表 .....           | 66 |

# 第一部分 项目总览与协作基础

## 第 1 章 项目架构与分工设计

### 1.1 你能学到什么

本章介绍整个项目的技术架构、模块划分和团队分工方式。读完本章，将帮助你理解一个多模块数据科学项目如何从零组织；各模块之间的数据流和依赖关系；如何在团队中分配任务而不产生耦合冲突。

### 1.2 项目背景

本项目旨在利用公开数据，构建一个"环境指纹—品种性状—管理方案"三要素互作模型。对标 Xiao et al. (2024, Nature Food 5:59-71) 的研究框架，我们使用世界气候数据(WorldClim)、土壤数据(SoilGrids, 中国土壤数据库)、地形数据(SRTM)、大气 CO2 浓度(CMIP6)和作物品种审定公告，构建约 **65 维输入特征矩阵**，通过 **XGBoost** 模拟 APSIM 过程模型输出，再使用 **NSGA-II** 搜索最优管理方案，并用 **SHAP 值**解释各特征对产量和环境指标的边际贡献。

项目的最终产出包括两套输出：最优农艺管理推荐方案（氮肥、灌溉、秸秆还田）；最优育种改良方向（哪些性状最值得改良，改良幅度与产量增益的关系）。

## 1.3 总体技术架构

整个项目按照数据流向分为五个层次：

**第一层：数据获取层。**负责从公开数据源下载原始栅格/表格数据，存放于 2026-SP/Data 目录下。涉及 Python 的 rasterio、urllib、BeautifulSoup 等库和手动浏览器下载。

**第二层：特征工程层。**将不同分辨率、不同投影的多源数据统一到 1km 网格，构建标准化的环境指纹矩阵。核心工具为 rasterio（重采样）、geopandas（空间连接）、pandas（表格操作）。

**第三层：建模层。**使用 XGBoost 回归器训练从环境指纹到产量/环境指标的映射模型。核心工具为 xgboost、scikit-learn（交叉验证、超参数调优）。

**第四层：优化层。**将训练好的 XGBoost 模型作为代理模型（surrogate），使用 NSGA-II 多目标遗传算法搜索最优的管理方案组合。核心工具为 pymoo。

**第五层：解释与部署层。**使用 SHAP 解释特征重要性，使用 Bootstrap 量化不确定性，使用 FastAPI 将模型部署为 RESTful 服务。

## 1.4 团队分工设计

建议按模块分为四个角色，每个角色可由 1-2 人承担：

| 角色 | 负责模块 | 核心技能 | 交付物 |
|----|------|------|-----|
|----|------|------|-----|



|       |                               |                                      |                       |
|-------|-------------------------------|--------------------------------------|-----------------------|
| 数据工程师 | 第 1-2 层：数据获取、预处理、特征工程         | rasterio, geopandas, pandas, xarray  | 环境指纹矩阵 (CSV/Parquet)  |
| 建模工程师 | 第 3-4 层：XGBoost 训练、NSGA-II 优化 | xgboost, scikit-learn, pymoo, optuna | 训练好的模型 (pkl)、优化结果 CSV |
| 分析工程师 | 第 5 层：SHAP 解释、Bootstrap、统计分析  | shap, numpy, scipy, R/tidyverse      | 特征重要性图、不确定性区间、统计检验报告  |
| 部署工程师 | 第 5 层：API 服务、可视化、文档           | FastAPI, Docker, GitHub Actions      | Docker 镜像、API 文档、使用手册 |

## 1.5 模块间接口约定

为避免模块间的耦合冲突，我们约定以下数据接口格式：所有模块间传递的数据均为 **CSV** 或 **Parquet** 格式的表格文件；每一行代表一个地理位置点（网格）；列名使用英文小写+下划线命名（snake\_case）；缺失值统一用 **NaN** 表示，不允许使用 -999 等魔法数字！；空间参考统一为 **WGS84(EPSG:4326)**，分辨率统一为 **0.008333 度（约 1km）**。

## 1.6 掌握后的延展能力

掌握了本章所述的项目架构设计方法后，你可以类推到任何以数据为中心的科研项目组织：多源遥感数据分析、环境监测系统、作物模型区域升尺度、以及任何需要"数据获取—特征工程—建模—优化—部署"五步链路的应用场景。这种分层架构是工业界数据科学团队的标准实践，可以说，掌握它就具备了在科技公司担任数据工程师或机器学习工程师的基础架构能力。

*当前业界趋势（2025-2026）*：模块化 ML 管道（Modular ML Pipelines）正成为主流。Kubeflow、MLflow、Metaflow 等工具本质上都是对这种分层架构的工程化实现。本项目使用"约定接口&独立模块"，与这些工业级工具的设计哲学一致。

## 第 2 章 Git 与 GitHub 团队协作

### 2.1 你能学到什么

本章介绍使用 **Git** 和 **GitHub** 进行团队代码管理的方法。读完本章，你将能够：创建和克隆代码仓库；使用分支(branch)进行独立开发而不影响他人；提交合并请求(Pull Request)并接受代码审查；解决合并冲突(merge conflict)；使用 GitHub Projects 或 Issues 跟踪任务进度。

### 2.2 为什么 Git 是必备技能

Git 是目前全球使用率最高的版本控制系统，2024 年 Stack Overflow 开发者调查显示超过 93% 的专业开发者使用 Git。在科研领域，Nature 和 Science 均建议论文代码通过 GitHub 发布以保证可重复性。对个人而言，GitHub profile 已成为科技行业求职的事实标准简历；对团队而言，Git 分支模型是多人协作而不产生代码冲突的唯一可靠方案。

### 2.3 项目仓库结构

推荐在 GitHub 上创建 Organization 或公共仓库，结构如下：

```
2026-SP/
├── .github/      # GitHub Actions CI/CD 配置
│   └── workflows/
├── data/         # 数据目录（gitignore，不上传大文件）
└── src/         # 源代码
```

```
| |— data_ingest/ # 数据获取脚本
| |— features/   # 特征工程
| |— models/     # 建模与训练
| |— optimization/ # NSGA-II 优化
| |— visualization/ # 可视化
|— tests/        # 单元测试
|— docs/         # 文档
|— notebooks/    # Jupyter 探索性分析
|— requirements.txt # Python 依赖
|— Dockerfile    # 容器化配置
|— README.md     # 项目说明
|— .gitignore    # 忽略文件列表
```

注：大文件（>100MB）如 GeoTIFF 不应提交到 Git，应使用 .gitignore 排除，通过 README 中的下载链接获取，或使用 Git LFS（Large File Storage）。

## 2.4 分支策略

推荐 GitHub Flow 模型，这是目前最简洁且工业界最广泛使用的分支策略：

**main 分支：**永远保持可运行状态，只通过 PR 合并，禁止直接 push。每次合并到 main 应触发自动测试。

**feature 分支：**从 main 分支创建，命名为 feature/功能描述，例如 feature/add-nsga2-optimizer、feature/shap-visualization。一个分支只做一件事，完成后发起 PR 合并回 main。

**fix 分支：**紧急修复分支，命名为 fix/问题描述，例如 fix/memory-leak-in-rasterio。修复后同时合并到 main。

## 2.5 日常工作流

每天开始工作时的标准操作：

```
git checkout main      # 切换到 main 分支
git pull origin main    # 拉取最新代码
git checkout -b feature/xxx # 创建新功能分支
```

完成一个功能后的提交流程：

```
git add .              # 暂存所有修改
git commit -m "feat: 添加 NSGA-II 优化模块" # 提交
git push origin feature/xxx # 推送到远程
```

然后在 GitHub 网页上创建 Pull Request，指定至少一位 reviewer。

审查通过后点击 Merge，最后删除远程分支：

```
git branch -d feature/xxx # 删除本地分支
git push origin --delete feature/xxx # 删除远程分支
```

## 2.6 提交信息规范

统一使用 Conventional Commits 格式，让提交历史清晰可读：

| 前缀       | 含义     | 示例                              |
|----------|--------|---------------------------------|
| feat     | 新功能    | feat: 添加 SHAP 特征重要性分析模块         |
| fix      | 修复 bug | fix: 修复 rasterio 内存泄漏问题         |
| docs     | 文档更新   | docs: 更新 API 接口文档               |
| refactor | 代码重构   | refactor: 将数据加载逻辑提取为独立模块        |
| test     | 测试相关   | test: 添加 XGBoost 交叉验证单元测试       |
| chore    | 杂项/构建  | chore: 更新 requirements.txt 依赖版本 |

## 2.7 掌握后的延展能力

掌握 Git/GitHub 工作流后，你可以参与任何开源项目（从提交第一个 PR 开始），理解 CI/CD 持续集成的基本概念，亦可为后续学习 DevOps 打基础。当前趋势（2025-2026）是"GitOps"——将 Git 作为基础设施配置和部署的唯一真相来源(source of truth)，Kubernetes 生态的 ArgoCD 和 Flux CD 都基于这一理念。你在这里学到的分支和 PR 流程，直接复用于这些前沿工具。

## 第 3 章 Python 开发环境与项目管理

### 3.1 你能学到什么

本章介绍如何搭建一个可复现的 Python 开发环境。你将学会使用 Conda 管理虚拟环境、使用 pip 安装依赖、理解 requirements.txt 的结构、配置 VS Code 进行高效开发。这些都是所有后续模块的基础。

### 3.2 环境隔离的重要性

Python 生态最大的痛点之一是**依赖冲突**：项目 A 需要 numpy 1.24，项目 B 需要 numpy 2.0，直接装在系统 Python 中必然导致其中一个项目报错。**虚拟环境（virtual environment）**为每个项目创建独立的 Python 解释器和包安装目录，彻底解决这一问题。

### 3.3 推荐环境方案

**方案一**：Miniconda + conda 环境（推荐新手使用）。安装 Miniconda（轻量版 Anaconda，约 50MB），然后：

```
conda create -n 2026sp python=3.11 # 创建环境
conda activate 2026sp             # 激活环境

pip install numpy pandas xarray rasterio geopandas xgboost shap scikit-learn pymoo optuna matplotlib
seaborn jupyter fastapi uvicorn docker

pip freeze > requirements.txt     # 锁定依赖版本
```

**方案二**：Python venv + pip（轻量方案）。适合已有 Python 3.11+的用户：

```
python -m venv venv          # 创建虚拟环境
venv\Scripts\activate       # Windows 激活
pip install -r requirements.txt # 安装所有依赖
```

### 3.4 项目文件组织规范

每个功能模块的 Python 文件应遵循以下规范：文件头包含**模块说明 docstring**；使用 `if __name__ == "__main__"` 保护可执行代码；将可配置参数抽取到 `config.py` 或 `config.yaml` 中；函数和变量命名使用 `snake_case`。

About `snake_case`（XX Case 一般指某种工作环境中的格式的规范，其不局限于编程领域，如我们可能涉及到的项目答辩中首页 PPT 呈现的项目标题，即为 Title Case）

`snake case`（蛇式）



snake case

特点：名称中间的标点被替换成下划线（`_`）。

如果所有单词都小写，称之为 `lower snake case`<sup>+</sup>（小蛇式），例如 `"get_user_name"`。

如果所有单词都大写，称之为 `upper snake case`<sup>+</sup>（大蛇式），例如 `"GET_USER_NAME"`。

示例：一个规范的特征工程模块开头：

```
"""
```

特征工程模块：从原始栅格数据构建环境指纹矩阵



**输入：** Data/WorldClim/\*.tif, Data/SRTM/\*.tif, ...

**输出：** features\_matrix.parquet

**用法：** python -m src.features.build\_matrix --config config.yaml

```
"""
```

```
import rasterio
```

```
import numpy as np
```

```
import pandas as pd
```

```
from pathlib import Path
```

```
def extract_features(grid_points, raster_dir):
```

```
    """从栅格数据中提取每个网格点的环境特征"""
```

```
    pass
```

```
if __name__ == "__main__":
```

```
    main()
```

### 3.5 掌握后的延展能力

虚拟环境和依赖管理是所有 Python 项目的起点。掌握了这些，你可以无缝切换到任何 Python 数据科学项目，包括深度学习（PyTorch/TensorFlow 环境）、Web 开发（Django/Flask 环境）、以及数据工程（Apache Spark/PySpark 环境）。当前趋势（2025-2026）是使用 uv 或 pixi 等新一代 Python 包管理器，它们比 pip 快

10-100 倍，但底层原理完全相同——你在这里学到的概念可以直接迁移。

## 第二部分 数据获取与预处理

### 第 4 章 地理空间数据处理

核心库：rasterio, xarray, geopandas, rioxarray

#### 4.1 你能学到什么

本章是项目的数据基础。你将学会：用 rasterio 读取和写入 GeoTIFF 格式的栅格数据；用 xarray 处理 NetCDF 格式的多维气候数据；用 geopandas 处理矢量边界（如平谷区行政边界）并进行空间连接；将不同分辨率、不同投影的数据统一重采样到同一网格。

#### 4.2 栅格数据的本质

GeoTIFF 文件本质上是一个二维（或三维）数字矩阵，附带了地理参考信息。地理参考信息包括：投影坐标系（CRS, Coordinate Reference System），定义了如何将经纬度映射到平面；仿射变换矩阵（Affine Transform），定义了像素坐标(row, col)与地理坐标(lon, lat)之间的线性转换关系；以及 NoData 值，标记无效区域。

当你用 rasterio.open() 打开一个 GeoTIFF 文件时，你获得的是一个类似 NumPy 数组的对象，但它知道自己在地球上的位置。这是地理空间数据分析的基本抽象。

#### 4.3 核心操作

操作一：读取栅格数据并获取元信息

```
import rasterio

with rasterio.open("wc2.1_2.5m_bio_1.tif") as src:

    print(f'CRS: {src.crs}')

    print(f'分辨率: {src.res}')

    print(f'尺寸: {src.width} x {src.height}')

    print(f'边界: {src.bounds}')

    data = src.read(1) # 读取第一波段
```

**操作二：按经纬度提取像素值。给定目标点的经纬度，将经纬度转换为像素行列号，再读取该位置的数值：**

```
from rasterio.transform import rowcol

row, col = rowcol(src.transform, lon, lat)

value = data[row, col] # 该位置的年均温
```

**操作三：重采样到统一分辨率。不同数据源的分辨率不同（WorldClim 约 5km，SRTM 约 30m），需要统一重采样到 1km 网格：**

```
from rasterio.enums import Resampling

# 计算 1km 分辨率下的新尺寸

new_height = int(src.height * src.res[1] / 0.008333)

new_width = int(src.width * src.res[0] / 0.008333)

resampled = src.read(1, out_shape=(new_height, new_width),
                           resampling=Resampling.bilinear)
```

## 4.4 与项目管理衔接

地理空间数据处理模块的输出是"每个目标网格点的特征值向量"。这个输出将被第 5 章的特征工程模块消费。因此，模块的输出格式约定为：一个 **Pandas DataFrame**，列包含 grid\_id、lon、lat、bio1、

bio2 ... 等；存储为 **Parquet** 格式（比 CSV 快 5-10 倍，且保留数据类型）；每个数据源单独一个文件，最后在第 5 章中合并。

在 GitHub 仓库中，数据处理脚本放在 `src/data_ingest/` 目录下。  
每个公开数据源对应一个子模块，例如：

`src/data_ingest/worldclim.py`、`src/data_ingest/soilgrids.py`、  
`src/data_ingest/srtm.py`。

## 4.5 常见问题与优化

内存溢出：读取全球范围的 GeoTIFF 可能导致内存不足。解决方案是使用窗口读取(windowed reading)，只加载目标区域：

```
from rasterio.windows import from_bounds
window = from_bounds(116.7, 40.0, 117.5, 40.5, src.transform)
subset = src.read(1, window=window)
```

投影不一致：不同数据源的 CRS 可能不同（WGS84 vs Goode Homolosine vs Albers），**必须统一转换**。rasterio.warp.reproject 可以将一个栅格重新投影到目标 CRS。

处理速度优化：对于大规模批量提取，不要逐个点循环读取，而应一次性读取整个目标区域的 **numpy** 数组，然后用向量化索引提取所有点。循环读取（10000 次 I/O 操作）可能耗时数分钟，而一次性读取+向量化索引只需几秒。

## 4.6 掌握后的延展能力

地理空间数据处理是遥感、GIS、环境科学和精准农业的核心技能。掌握了 `rasterio/geopandas/xarray` 这套工具链后，你可以处理任何卫星遥感数据（Landsat、Sentinel-2、MODIS）、气候再分析数据（ERA5、NCEP）、以及数字土壤制图产品。当前趋势（2025-2026）是"云原生地理空间"（Cloud-Native Geospatial）：STAC（SpatioTemporal Asset Catalog）规范和 COG（Cloud Optimized GeoTIFF）格式使得无需下载完整文件即可在云端查询和读取特定时空范围的数据。你在这里学到的 `rasterio` 窗口读取技术，正是这一趋势的基础。

## 第 5 章 多源数据融合与特征工程

核心库：pandas, numpy, scipy

### 5.1 你能学到什么

本章教你如何如何将多个不同来源、不同格式的数据融合为统一的分析就绪表格。你将掌握：Pandas DataFrame 的高级操作

（merge/join/groupby/transform）；处理缺失值的多种策略（删除、均值填充、KNN 填充、MICE 多重插补）；特征缩放与标准化

（StandardScaler、MinMaxScaler）；衍生特征的计算（从已有特征生成新特征）。

### 5.2 为什么特征工程决定模型上限

数据科学界有一句名言："数据和特征决定了机器学习的上限，而模型和算法只是在逼近这个上限。"在环境-作物建模中亦如此：如果土壤有机碳数据缺失或错误，无论用多复杂的模型都无法正确预测产量。特征工程是连接"原始数据"和"可用模型输入"之间的桥梁，也是项目中最耗时但最重要的环节（通常占整个项目时间的 60-70%）。

### 5.3 数据融合流程

本项目的多源数据融合预计分为四步：

**第一步：统一空间网络。**以平谷区行政边界为掩膜，生成间距约 1km 的规则网格点（约 65 个点），每个点用(lon, lat)标识。所有后续提取均基于这个网格。

**第二步：逐源提取特征。**对每个网格点，从各个栅格数据源中提取对应的数值：从 WorldClim bio tif 中提取 19 个生物气候变量；从 WorldClim monthly tif 中提取 3-9 月的月均温和月降水（14 维）；从 SRTM tif 中提取海拔、坡度和坡向（3 维）；从 SoilGrids VRT 中提取砂粒、粉粒、黏粒、有机碳等（5 维已获取+3 维待补充）；从 CO2 查表中按年份匹配浓度（1 维）。

**第三步：衍生特征计算。**从月温数据中计算生长期日（GDD, Growing Degree Days）：对每个基温阈值（0 度 C、5 度 C、10 度 C、15 度 C、20 度 C、25 度 C），模拟逐日温度曲线（三角函数插值）并累加超过阈值的热量单位。GDD 是作物模型中最重要的热时间指标。另从 CHELSA 逐日数据中计算极端气候指标：生育期 Tmax 的 95 分位数（热胁迫）、Tmin 的 5 分位数（冷胁迫）、降水的 95 分位数（涝渍风险）。

**第四步：合并与清洗。**将所有提取结果按(grid\_id, year)合并为一张大表，处理缺失值，检查异常值（如海拔为负在平谷区不合理），进行标准化。



## 5.4 核心代码示例

多源数据合并（merge）的核心操作：

```
import pandas as pd

# 加载各数据源（均按 grid_id 索引）
df_bio = pd.read_parquet("features_bio.parquet")
df_clim = pd.read_parquet("features_clim.parquet")
df_soil = pd.read_parquet("features_soil.parquet")
df_terrain = pd.read_parquet("features_terrain.parquet")

# 链式合并
df_all = (df_bio
          .merge(df_clim, on=["grid_id", "year"])
          .merge(df_soil, on="grid_id")
          .merge(df_terrain, on="grid_id")
          )
```

GDD 计算（从月均温通过三角函数插值得到逐日温度，再累加超过基温的部分）：

```
import numpy as np

def calc_gdd(monthly_tavg, base_temp):
    """从 12 个月的月均温计算生长度日"""
    # 三角函数重建逐日温度（Forsythe et al. 1995 方法）
    days_per_month = [31,28,31,30,31,30,31,31,30,31,30,31]
    daily_temps = []
    for m in range(12):
        # 月内每日温度正弦插值
        for d in range(days_per_month[m]):
```

```
frac = (d + 0.5) / days_per_month[m]
daily_temps.append(monthly_tavg[m])
daily_temps = np.array(daily_temps)
return np.maximum(daily_temps - base_temp, 0).sum()
```

## 缺失值处理的三层策略：

```
from sklearn.impute import SimpleImputer, KNNImputer
```

```
# 策略 1: 若缺失率 < 1%, 中位数填充
```

```
imputer = SimpleImputer(strategy="median")
```

```
# 策略 2: 若缺失率 1-10%, KNN 填充（利用相邻网格相关性的填充）
```

```
imputer = KNNImputer(n_neighbors=5)
```

```
# 策略 3: 若缺失率 > 10%, 该特征考虑剔除或在论文中明确讨论
```

## 5.5 掌握后的延展能力

特征工程是数据科学最通用的技能，几乎适用于任何领域。你在这里学到的 **pandas** 链式操作（**merge-transform-groupby** 流水线）、缺失值处理策略、衍生特征构建方法，在以下领域完全复用：金融风控（交易特征构建）、推荐系统（用户行为特征）、医疗诊断（电子病历特征）、NLP（文本特征提取）。当前趋势（2025-2026）是"特征存储"（**Feature Store**）的兴起——像 **Feast** 和 **Tecton** 这样的工具将特征工程标准化为可共享、可版本化的资产。你在这里学到的"特征矩阵"概念，正是 **Feature Store** 的核心数据模型。

## 第 6 章 环境指纹构建

核心库: rasterio, numpy, pandas, scipy

### 6.1 你能学到什么

本章将第 4 章和第 5 章的技能串联起来, 完成项目中最核心的数据产物: "环境指纹矩阵"。你将学会: 如何将 20 余个独立的 GeoTIFF 文件转化为一张规整的分析表; 如何设计一个可复用的特征提取流水线; 如何验证提取结果的空间一致性和物理合理性。

### 6.2 什么是"环境指纹"

"环境指纹" (Environmental Fingerprint) 是我们在本项目中的核心概念。它为每一个地理位置创建一个独特的"身份标识"——由 55-75 个数值组成的向量, 完整地刻画该位置的气候、土壤、地形和大气条件。可以类比于: 人类的指纹由几十个特征点唯一标识一个人; 农田的"指纹"由几十个环境变量唯一标识一块地。

这个概念对应 Xiao et al. (2024) 论文中的"环境特征空间"。在论文中, 作者用约 30-35 个环境变量训练 XGBoost 模型来模拟 APSIM 的输出。我们的"指纹"在此基础上扩展了品种性状维度和更精细的土壤属性, 力图提供更全面的环境刻画。

### 6.3 结构程序设计

特征提取流水线应设计为模块化的 Pipeline 类:

```
class EnvironmentalFingerprintBuilder:
```

"""环境指纹构建器：从栅格数据中提取每个网格点的特征向量"""

```
def __init__(self, config):
    self.data_dir = Path(config["data_dir"])
    self.grid_points = self._load_grid(config["grid_file"])
    self.target_crs = "EPSG:4326"
```

```
def _load_grid(self, grid_file):
    """加载目标网格点（平谷区 1km 网格）"""
    return pd.read_parquet(grid_file)
```

```
def extract_worldclim_bio(self):
    """提取 19 个生物气候变量"""
    bio_dir = self.data_dir / "WorldClim"
    features = {}
    for i in range(1, 20):
        tif_path = bio_dir / f"wc2.1_2.5m_bio_{i}.tif"
        with rasterio.open(tif_path) as src:
            for idx, row in self.grid_points.iterrows():
                features.setdefault(f"bio {i}", []).append(
                    src.sample([(row["lon"], row["lat"])]).item())
    return pd.DataFrame(features)
```

```
def extract_all(self):
    """执行完整提取流水线"""
    df_bio = self.extract_worldclim_bio()
    df_clim = self.extract_monthly_climate()
    df_terrain = self.extract_terrain()
    df_soil = self.extract_soil()
    df_co2 = self.extract_co2()
```

```
return pd.concat([df_bio, df_clim, df_terrain, df_soil, df_co2], axis=1)
```

## 6.4 质量控制

每条特征提取完成后，必须进行以下验证：

**数值范围检查：**检查提取值是否在物理合理范围内（例如平谷区年均温应在 5-15 度 C 范围内，年降水应在 300-800mm 范围内）。超出范围的值可能指示坐标错误或数据源问题。

**空间一致性检查：**相邻网格点的特征值应该是渐变的。如果某个网格点的 bio1（年均温）比相邻点高 5 度 C 以上，可能有异常。可以计算变异函数(variogram)或简单的相邻点差值来检测。

**缺失网格检查：**确保每个目标网格点都有对应的特征值。如果某些点在数据源的范围之外（如边界网格），应该在输出中明确标注。

## 6.5 掌握后的延展能力

"环境指纹"这个概念可以直接推广到以下场景：任何需要"将地理位置转化为固定维度特征向量"的空间预测任务（如物种分布建模、土壤碳储量预测、城市热岛分析）；精准农业中的管理区划（Management Zone Delineation）——将农田按环境指纹聚类，对不同的管理区采用不同方案。当前趋势（2025-2026）是"数字孪生"（Digital Twin）在农业中的应用：为每一个田块创建一个数字副

本，持续更新其状态。环境指纹是数字孪生的静态部分，搭配时序遥感数据后即构成完整的数字孪生数据层。

## 第三部分 建模与优化

### 第 7 章 机器学习建模：从决策树到 XGBoost

核心库：scikit-learn, xgboost, optuna

#### 7.1 你能学到什么

本章是项目的核心建模部分。你将学会：理解决策树的分裂原理（信息增益、基尼系数）；理解集成学习为什么优于单棵树（**Bagging** 与 **Boosting** 的区别）；掌握 **XGBoost** 的梯度提升机制及其关键超参数；使用交叉验证评估模型性能；使用 **Optuna** 进行自动超参数调优。

#### 7.2 从决策树到梯度提升：完整推导

第一步：决策树的直觉。假设你要预测某块地的玉米产量，你手上有年均温、年降水、土壤有机碳三个特征。决策树的思路是不断地问"是/否"问题来将数据分组：年均温是否大于 12 度 C？如果是，分到左枝；年降水是否大于 500mm？如果是，再分到左枝...**每次分裂选择那个能让两组产量差异最大化的特征和阈值。**在数学上，这个"差异"用信息增益或基尼系数衡量——分裂后子节点的"纯度"减去分裂前的"纯度"。

第二步：单棵树的致命缺陷。单棵决策树极易过拟合：它可以把训练数据分到每个叶子只有一个样本，在训练集上完美但在测试

集上一塌糊涂。想象你背下了一张考试卷的所有答案——你在这张卷子上满分，但换一套卷子就全错了。这就是过拟合。

第三步：集成学习的思想。与其依赖一棵"独裁者"树，不如让一群树民主投票。Bagging (**Bootstrap Aggregating**，如随机森林)的思路是：并行训练 **100** 棵树，每棵用不同的数据子集和特征子集，最后取所有树的平均预测值。这能大幅降低方差。

第四步：Boosting 的革命。Bagging 的 100 棵树是独立训练的，每棵树不知道其他树在做什么。Boosting (提升法，如 XGBoost) 改变了这个范式：树是串行训练的，每一棵新树的使命是修正前面所有树的残差（误差）。想象你在学射箭：第一箭偏左了 5 厘米——第二箭你就专门瞄准"比目标偏右 5 厘米"的位置——第三箭再修正第二箭的偏差...最终，所有箭的"合力"会非常接近靶心。这就是梯度提升的直觉：每一棵新树拟合的是前序模型预测值与真实值之间的负梯度（即残差）。

第五步：XGBoost 的工程优化。传统的梯度提升（如 sklearn 的 GradientBoostingRegressor）每次分裂需要遍历所有特征的所有可能阈值，计算量巨大。XGBoost 引入了三项关键优化：加权分位数草图（Weighted Quantile Sketch），将连续特征离散化为分位数桶，大幅减少候选分裂点；缓存感知访问（Cache-aware Access），利用 CPU 缓存结构加速数据读取；正则化项引入叶节点权重和树深度惩



罚，天然防过拟合。这些优化使得 XGBoost 比传统的 GBRT 快 10 倍以上，同时精度更高。

### 7.3 本项目中的建模设计

目标变量：本项目需要训练多个 XGBoost 模型。小麦产量模型，输入环境指纹+品种性状+管理参数，输出小麦产量；玉米产量模型，同理；氮淋溶模型，输出  $\text{NO}_3\text{-N leaching}$  量（氮淋溶属于土壤淋溶作用的一种，主要表现为土壤中的可溶性氮素（如硝态氮和铵态氮）在降雨或灌溉水作用下，从土壤上层向下层迁移，部分氮素可能随地下水流失，导致土壤肥力下降和水体污染风险增加。）；土壤有机碳变化模型，输出  $\Delta\text{SOC}$ 。

训练-验证-测试划分策略：由于空间自相关的存在（相邻网格的环境非常相似），不能简单地随机划分。必须使用空间分块交叉验证（**Spatial Block CV**）：将平谷区的 65 个网格按地理位置分为 5 块，每次用 4 块训练、1 块验证，旋转 5 次。这样可以检测模型能否泛化到未见过的地理位置，而不是靠空间邻近性作弊。

### 7.4 核心代码

```
import xgboost as xgb

from sklearn.model_selection import cross_val_score

import optuna

# XGBoost 基础训练
model = xgb.XGBRegressor(
```

```

n_estimators=500,    # 树的数量
max_depth=6,        # 每棵树的最大深度
learning_rate=0.05,  # 学习率（每一棵树的"步伐"大小）
subsample=0.8,       # 每棵树随机使用 80%训练样本
colsample_bytree=0.8, # 每棵树随机使用 80%特征
reg_alpha=0.1,       # L1 正则化（稀疏性）
reg_lambda=1.0,      # L2 正则化（权重衰减）
random_state=42
)
model.fit(X_train, y_train)
score = model.score(X_test, y_test) # R-squared

# Optuna 自动超参数调优
def objective(trial):
    params = {
        "max_depth": trial.suggest_int("max_depth", 3, 10),
        "learning_rate": trial.suggest_float("lr", 0.01, 0.3),
        "subsample": trial.suggest_float("subsample", 0.5, 1.0),
        "colsample_bytree": trial.suggest_float("colsample", 0.5, 1.0),
        "reg_alpha": trial.suggest_float("alpha", 1e-8, 1.0),
        "reg_lambda": trial.suggest_float("lambda", 1e-8, 1.0),
    }
    model = xgb.XGBRegressor(**params, n_estimators=500,
                             early_stopping_rounds=20)
    scores = cross_val_score(model, X_train, y_train, cv=5,
                             scoring="r2")
    return scores.mean()

study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=50)

```

```
print(f'最佳超参数: {study.best_params}')
```

## 7.5 训练技巧与常见陷阱

**早停（Early Stopping）：**训练过程中如果在验证集上连续 20 轮没有提升，自动停止。这能防止过拟合并节省训练时间。

**特征重要性偏差：**XGBoost 默认的 `feature_importances_` 偏向高基数特征和有更多分裂机会的特征。不要直接用它下结论，应配合第 8 章的 SHAP 分析进行交叉验证。

**样本量不足的处理：**平谷区仅约 65 个 1km 网格，对于 500 棵树的 XGBoost 可能不够。解决方案包括：从周边区县扩展更多网格（顺义、密云、三河）；使用 CMIP6 未来气候数据进行数据增强（将同一位置的不同年份/情景视为独立样本）。

## 7.6 掌握后的延展能力

XGBoost 是结构化数据（表格数据）建模的王者，2015-2023 年间赢得了绝大多数 Kaggle 竞赛。掌握它后，你可以：直接参与任何表格数据相关的机器学习竞赛或项目；理解几乎所有集成学习方法的原理（LightGBM、CatBoost、随机森林）；为学习深度学习打下基础（梯度提升中的“梯度”与神经网络反向传播中的“梯度”在数学上是同一个概念）。当前趋势（2025-2026）：XGBoost 仍然是表格数据的首选模型，但神经网络的 TabTransformer 和 FT-Transformer

等架构正在追赶。你在这里学到的梯度提升原理是永恒的，无论工具如何演进。

## 第 8 章 模型解释：SHAP 值原理与应用

核心库：shap, matplotlib, seaborn

### 8.1 你能学到什么

本章教你"打开黑箱"：理解机器学习模型为什么做出某个预测。你将学会：Shapley 值的博弈论起源及其公平分配原理；使用 SHAP 的 TreeExplainer 高效解释 XGBoost 模型；绘制 **Summary Plot**（蜂群图）和 **Dependence Plot**（依赖图）；计算特征交互效应。

### 8.2 Shapley 值的直觉理解

Shapley 值来自合作博弈论（Lloyd Shapley, 1953 年，2012 年诺贝尔经济学奖）。核心问题：一个团队完成了一个项目获得了一笔奖金，如何公平地将奖金分配给每个成员？公平的定义是：每个人的份额应等于他们加入团队时带来的边际贡献。

SHAP（SHapley Additive exPlanations）将这个想法移植到机器学习中："团队"是所有输入特征，"奖金"是模型的预测值，"每个人"是单个特征。SHAP 值回答：特征 X 对这个预测值的贡献是多少？

**例如：**模型预测某网格的小麦产量为 6.2 吨/公顷（而所有网格的平均产量是 5.5 吨/公顷）。**SHAP 分解：**年均温 10.5 度 C 贡献了 +0.3 吨（正面），土壤有机碳 1.2% 贡献了 +0.5 吨（正面），年降水 620mm 贡献了 -0.1 吨（略负面，可能过湿）。总和： $5.5 + 0.3 + 0.5 - 0.1 = 6.2$ 。这就是可解释的 AI。

## 8.3 核心代码

```
import shap

# 使用 TreeExplainer 获取 SHAP 值
explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_test)

# Summary Plot: 全局特征重要性
shap.summary_plot(shap_values, X_test, feature_names=feature_names)

# Dependence Plot: 单个特征与 SHAP 值的关系
shap.dependence_plot("bio1", shap_values, X_test)

# 单个预测的瀑布图（解释一个特定网格的预测）
shap.waterfall_plot(explainer.expected_value,
                    shap_values[0], X_test.iloc[0])
```

## 8.4 SHAP 在育种中的应用

本项目中 SHAP 的独特应用是"性状缺口分析"（Trait Gap Analysis）：在品种性状特征中，**哪些性状对产量缺口的贡献最大？**如果 SHAP 分析显示"千粒重"对产量的边际贡献为+0.5 吨/公顷（在现有遗传背景下），而审定的新品种最高千粒重可提高 5 克，那么可以预期产量增益约为 0.5 吨/公顷——这就是育种方向的定量预测。

## 8.5 掌握后的延展能力

可解释 AI (XAI) 是 2025-2026 年 AI 领域最热门的子方向之一。欧盟 AI 法案 (EU AI Act, 2024 年通过) 要求高风险 AI 系统必须提供可解释性。掌握 SHAP 后, 你可以在任何领域产出可信赖的模型解释: 金融信贷决策 (为什么拒绝这笔贷款)、医疗诊断 (为什么判断为高风险)、政策评估 (哪些因素驱动了政策效果)。当前趋势是 SHAP 与 LLM (大语言模型) 的结合——让 LLM 用自然语言描述 SHAP 分析结果, 形成自动生成的模型分析报告。

## 第 9 章 多目标优化：NSGA-II

核心库：pymoo

### 9.1 你能学到什么

本章介绍如何使用遗传算法解决多目标优化问题。你将学会：  
理解帕累托最优（Pareto Optimality）的概念；掌握 NSGA-II 算法的工作原理（非支配排序+拥挤度距离）；使用 pymoo 库定义优化问题并求解；理解约束优化与无约束优化的区别。

### 9.2 为什么多目标优化在农业中如此重要

农业生产本质上是一个多目标权衡问题。农民希望同时实现：  
产量最大化（经济目标），氮肥用量最小化（成本目标），氮淋溶和水消耗最小化（环境目标）。这些目标往往是冲突的——多施氮肥能增产，但增加了淋溶风险和环境成本。*单目标优化（如"最大化产量"）给出的答案是施肥无限大——这毫无实际意义。*多目标优化找到的是**帕累托前沿**：在所有可能的方案中，无法在不损害至少一个目标的情况下改善另一个目标的方案的集合。

### 9.3 NSGA-II 算法原理

NSGA-II（Non-dominated Sorting Genetic Algorithm II, Deb et al. 2002）是目前最广泛使用的多目标进化算法。它的工作流程模拟了自然选择：



**初始化：**随机生成  $N$  个候选管理方案（种群），每个方案是一个参数向量（如[氮肥量, 灌溉量, 秸秆还田率]）。

**评估：**用训练好的 **XGBoost** 模型预测每个方案的产量和环境影响（代理评估，比跑 APSIM 快 10000 倍）。

**选择：**非支配排序将所有方案分为多个前沿层级。**第一前沿（Pareto Front）**中的方案彼此非支配——没有一个方案在所有目标上都优于另一个。在同一个前沿内，拥挤度距离（**Crowding Distance**）鼓励选择与邻居差异大的方案，以保持解的多样性。

**变异与交叉：**选中的优秀方案通过交叉（交换参数）和变异（随机微调参数）产生下一代方案。

**迭代：**重复"评估-选择-交叉-变异"数百代，最终收敛到近似的帕累托前沿。

## 9.4 核心代码

```
import numpy as np
from pymoo.algorithms.moo.nsga2 import NSGA2
from pymoo.core.problem import Problem
from pymoo.optimize import minimize
from pymoo.operators.crossover.sbx import SBX
from pymoo.operators.mutation.pm import PM
from pymoo.operators.sampling.rnd import FloatRandomSampling
```

```
class FarmManagementProblem(Problem):
```

```
    """农田管理多目标优化问题"""
```

```

def __init__(self, xgb_model, env_features):
    # 3 个决策变量: 氮(kg/ha)、灌溉(mm)、秸秆还田(%)
    super().__init__(
        n_var=3, n_obj=2, n_constr=0,
        xl=np.array([50, 0, 0]), # 下界
        xu=np.array([300, 600, 100]) # 上界
    )
    self.model = xgb_model
    self.env = env_features

def _evaluate(self, x, out, *args, **kwargs):
    # x: (n_population, 3) 管理参数矩阵
    # 将管理参数与环境特征拼接为完整输入
    n = x.shape[0]
    X_full = np.hstack([
        np.tile(self.env, (n, 1)), # 环境特征
        x # 管理参数
    ])
    preds = self.model.predict(X_full)
    # 目标 1: 产量 (最大化 -> 取负值用于最小化)
    # 目标 2: 氮淋溶 (最小化)
    out["F"] = np.column_stack([-preds[:, 0], preds[:, 1]])

problem = FarmManagementProblem(xgb_model, env_sample)
algorithm = NSGA2(
    pop_size=100,
    sampling=FloatRandomSampling(),
    crossover=SBX(prob=0.9, eta=15),
    mutation=PM(prob=1/3, eta=20),
)

```

```
res = minimize(problem, algorithm, ("n_gen", 200))  
print(f'帕累托前沿方案数: {len(res.F)}')
```

## 9.5 NSGA-II 的局限性及本项目中的应对

直接使用 NSGA-II 搜索 66 维输入空间（~60 维环境指纹 + 3 维管理参数 + ~3 维品种性状）是不可行的——"维数灾难"会使进化算法在可行时间内无法收敛。Xiao et al. (2024)的解决方案是先训练 XGBoost 模型作为 APSIM 的代理（surrogate），然后只对管理参数进行优化（环境指纹固定为特定位置的观测值）。我们采用相同策略：对于每个目标网格点，固定其环境指纹和品种性状，只搜索最优管理参数。

备选方案：如果 NSGA-II 的代理模型方法仍然过于复杂，可以退化为使用论文已发表的优化管理方案作为标签，训练一个 MultiOutputRegressor 直接预测最优管理参数。这是一种更简单的监督学习方法，适合初期快速出结果。

## 9.6 掌握后的延展能力

多目标优化是工程设计和决策支持的核心技术。掌握 NSGA-II 后，你可以解决任何需要"在多个冲突目标中找到最佳折衷"的问题：供应链优化（成本 vs 时效 vs 碳排放）、建筑能效设计（建设成本 vs 运营能耗）、药物分子设计（药效 vs 毒性 vs 合成难度）。当前趋势（2025-2026）是基于强化学习的多目标优化——使用神经网络替代进化算法，能更高效地探索高维决策空间。你在这里学到的帕

累托最优概念，同样是强化学习的多目标版本（Multi-Objective RL）的基础。

## 第 10 章 不确定性量化：Bootstrap 方法

核心库：numpy, scipy, scikit-learn

### 10.1 你能学到什么

本章教你量化和表达模型预测的不确定性。你将学会：理解偶然不确定性(aleatoric)与认知不确定性(epistemic)的区别；使用 **Bootstrap** 重采样估计预测的置信区间；理解为什么"点估计"（只给出一个数字）在科研和决策中是不够的。

### 10.2 为什么不确定性量化至关重要

假设模型预测某网格的最优施氮量为 180 kg/ha，产量为 7.2 吨/ha。如果是你自己家的地，你会严格按照 180 kg/ha 施肥吗？你一定会问：这个预测有多可靠？ $\pm 5$  kg/ha 还是  $\pm 50$  kg/ha？如果有 50% 的概率实际产量在 6.0-6.5 吨之间，你可能会选择更保守的方案。这就是不确定性量化的意义——它为决策提供风险信息。

在论文中，报告预测值而不报告置信区间是不完整的。模型预测的不确定性，不同 CMIP6 气候情景下的预测范围，都需要置信区间作为支持，因此要提前做好不确定性量化。

### 10.3 Bootstrap 原理

Bootstrap（自助法，Efron 1979）是统计学中最优雅的想法之一。核心思路：我们只有一份样本（ $n=65$  个网格），但我们想知道如果重新采样，模型的预测会如何变化。Bootstrap 的解决方案是："

假装"手中的样本就是总体，从中有放回地重采样  $n$  次，形成一个新的 **Bootstrap** 样本（有些网格被抽到多次，有些一次也没有），在这个新样本上重新训练模型并做出预测。重复这个过程 1000 次，就得到了 1000 个可能的世界中的预测值——这些值的分布就是不确定性。

## 10.4 核心代码

```
import numpy as np
from sklearn.utils import resample

def bootstrap_predictions(model, X, y, X_pred, n_bootstrap=1000):
    """Bootstrap 预测：生成预测值的分布"""
    preds = np.zeros((n_bootstrap, X_pred.shape[0]))

    for i in range(n_bootstrap):
        # 有放回重采样训练数据
        X_bs, y_bs = resample(X, y, n_samples=len(X))

        # 在 Bootstrap 样本上训练
        model.fit(X_bs, y_bs)

        # 对目标位置做预测
        preds[i] = model.predict(X_pred)

    # 计算统计量
    mean_pred = preds.mean(axis=0)
```

```
lower_ci = np.percentile(preds, 2.5, axis=0) # 95% CI 下界  
upper_ci = np.percentile(preds, 97.5, axis=0) # 95% CI 上界
```

```
return mean_pred, lower_ci, upper_ci
```

## 10.5 不确定性分解

总预测不确定性可以分解为两个来源：

数据不确定性（偶然不确定性）：由测量误差、空间变异等引起。例如同一个网格内不同年份的产量波动。这种不确定性是固有的，无法通过增加数据来消除。

模型不确定性（认知不确定性）：由训练数据不足、模型结构选择等引起。例如只有 65 个训练样本时，模型对不同超参数组合给出不同的预测。这种不确定性可以通过增加数据、改进模型来降低。

在论文中，建议将这两种不确定性分别报告，以体现科学严谨性。Bootstrap 方法本身主要量化模型不确定性。数据不确定性可以额外通过 XGBoost 的 quantile regression 或 Monte Carlo Dropout 的变体来估计。

## 10.6 掌握后的延展能力

不确定性量化是所有科学计算和工程决策的基础。掌握了 Bootstrap 和置信区间后，你可以：在任何统计建模中提供误差棒和置信区间；理解贝叶斯统计的基本思想（Bootstrap 是频率学派的方

法，贝叶斯用后验分布表达不确定性，两者在概念上互补）；参与需要风险评估的场景（临床试验、金融风控、结构工程设计）。当前趋势（2025-2026）是"符合性预测"（Conformal Prediction）的兴起——它提供分布无关(distribution-free)的预测区间保证，比Bootstrap更适用于小样本深度学习模型。



## 第四部分 R 语言统计与可视化

### 第 11 章 R 语言统计分析

核心包：tidyverse (dplyr, tidyr, ggplot2), caret, randomForest, vegan

#### 11.1 你能学到什么

本章教你使用 R 语言进行统计分析和建模。你将学会：R 的 tidyverse 数据操作范式（管道操作符 %>%）；使用 R 进行方差分析 (ANOVA)、多元回归、混合效应模型等经典统计；理解何时用 R 优于 Python（探索性数据分析、统计检验、学术图表）。

#### 11.2 为什么项目需要 R

Python 在机器学习和工程方面更强，但 R 在统计推断和学术出版方面有独特优势：R 的统计检验报告比 Python 更完备（效应量、假设检验诊断图自动生成）；ggplot2 公认是科技论文出版质量的唯一标准绘图系统；R 的生态学/农学专用包（vegan 用于群落分析，agricolae 用于田间试验设计，metan 用于多环境试验分析）在 Python 中没有等价物。

在本项目中，R 的角色是 Python 的互补，而非替代。Python 处理大规模数据提取、模型训练、优化搜索；R 处理统计分析、假设检验、论文图表制作。两者通过 CSV 文件在文件系统层面交互。

## 11.3 R 与 Python 的互操作

现代数据科学工作流程中，R 和 Python 并不冲突。以下三种方式可以实现互操作：

方式一（最简单）：通过 CSV 文件传递。Python 写出结果到 CSV，R 读入做统计和绘图。这是本项目推荐的方式。

方式二：通过 `reticulate` 包在 R 中调用 Python。在 R 脚本中 `import Python` 模块、调用 Python 函数。适合需要频繁交互的场景。

方式三：通过 Quarto 或 R Markdown。在同一个 .qmd 文件中混合编写 Python 和 R 代码块，生成统一的 HTML/PDF 报告。Quarto 是 RStudio 开发的下一代科学出版系统，推荐用于最终报告生成。

## 11.4 核心操作示例

使用 `tidyverse` 进行数据操作（管道流）：

```
library(tidyverse)

# 读取 Python 输出的特征矩阵
df <- read_csv("features_matrix.csv")

# 管道操作：筛选 -> 选择列 -> 分组 -> 汇总
df %>%
  filter(year == 2020) %>%
  select(grid_id, bio1, bio12, yield) %>%
  group_by(cut(bio12, breaks = 5)) %>% # 按降水五分位分组
  summarise(
```

```
mean_yield = mean(yield, na.rm = TRUE),  
sd_yield = sd(yield, na.rm = TRUE),  
n = n()  
)
```

方差分析（ANOVA）检验不同管理方案之间的产量差异是否显著：

```
# 单因素 ANOVA  
model <- aov(yield ~ nitrogen_level, data = df)  
summary(model)  
TukeyHSD(model) # 事后两两比较  
  
# 双因素 ANOVA（氮水平 x 灌溉水平交互效应）  
model2 <- aov(yield ~ nitrogen_level * irrigation_level, data = df)  
summary(model2)
```

## 11.5 掌握后的延展能力

R 语言是统计学的通用语言，在以下领域有不可替代的作用：生物统计学（临床试验、流行病学）、生态学（群落分析、物种分布模型）、数量遗传学（GWAS、基因组选择）、社会科学（结构方程模型、社会网络分析）。当前趋势（2025-2026）是 Posit（原 RStudio）公司推动的"多语言数据科学平台"——在同一个 IDE 中同时使用 R、Python、Julia 和 SQL。你在这里掌握的双语言能力正是这一趋势的核心竞争力。

## 第 12 章 R 语言科学可视化

核心包：ggplot2, patchwork, ggsci, ggrepel, sf, tmap

### 12.1 你能学到什么

本章教你使用 ggplot2 制作出版级别的科学图表。你将学会：ggplot2 的分层语法（Grammar of Graphics）——将图表分解为数据、几何对象、坐标系、分面和主题五个独立层次；制作论文中常见的图表类型：箱线图、散点图矩阵、热图、地图；使用 patchwork 组合多张图为一张复合图；使用 ggsci 应用学术期刊的配色方案（Nature、Science、Lancet 风格）。

### 12.2 ggplot2 语法哲学

ggplot2 的核心思想是"图形语法"（Grammar of Graphics, Wilkinson 2005）：任何统计图形都可以分解为以下组件的组合：data（要可视化的数据框）、aesthetics（美学映射——哪个变量映射到 x 轴、y 轴、颜色、大小）、geometries（几何对象——点、线、柱、箱）、facets（分面——按分类变量拆分为子图）、themes（主题——字体、背景、网格线）。这种分层设计使得 ggplot2 比 matplotlib 更易于构建复杂图表：你不需要计算每个点的像素坐标，只需要声明变量之间的映射关系。

### 12.3 核心代码示例

```
library(ggplot2)
```

```
library(patchwork)
```

```
library(ggsci)
```

```
# 图 1: SHAP 特征重要性 (蜂群图用 geom_point 替代)
```

```
p1 <- ggplot(shap_df, aes(x = reorder(feature, abs_shap), y = shap_value, color = feature_value)) +  
  geom_jitter(alpha = 0.6, width = 0.2) +  
  coord_flip() +  
  scale_color_gradient2(low = "blue", mid = "grey", high = "red") +  
  labs(x = "", y = "SHAP value", title = "特征重要性 (SHAP)") +  
  theme_minimal()
```

```
# 图 2: 帕累托前沿散点图
```

```
p2 <- ggplot(pareto_df, aes(x = yield, y = n_leaching, color = irrigation)) +  
  geom_point(size = 3, alpha = 0.8) +  
  scale_color_viridis_c() +  
  labs(x = "产量 (t/ha)", y = "氮淋溶 (kg/ha)",  
       title = "帕累托前沿: 产量 vs 环境代价") +  
  theme_classic()
```

```
# 图 3: 不确定性带 (Bootstrap CI)
```

```
p3 <- ggplot(ci_df, aes(x = n_level, y = yield_pred)) +  
  geom_ribbon(aes(ymin = ci_lower, ymax = ci_upper), alpha = 0.3) +  
  geom_line(size = 1, color = "steelblue") +  
  labs(x = "施氮量 (kg/ha)", y = "预测产量 (t/ha)")
```

```
# 使用 patchwork 组合三张图
```

```
combined <- (p1 | p2) / p3 +  
  plot_annotation(tag_levels = "a") # 自动添加(a)(b)(c)标签  
ggsave("figure_combined.pdf", combined, width = 12, height = 8)
```

## 12.4 图表规范

制作论文级别的图表时需遵循以下规范：所有文字使用 Times New Roman 字体（与正文一致）；配色使用色盲友好的调色板（`viridis / cividis / ggsci::scale_color_npg`）；字号不小于 8pt，确保印刷后可读；图表标签使用(a)(b)(c)而非 1,2,3（遵循大多数期刊规范）；导出为 PDF 或 600 DPI 的 TIFF 格式（而非 PNG/JPG）。

## 12.5 掌握后的延展能力

`ggplot2` 是数据可视化领域的标准工具，掌握后你可以制作任何领域的出版级图表。当前趋势（2025-2026）有两个方向：交互式可视化（`plotly`、`gganimate`）将静态图表变为可交互的 Web 应用；AI 辅助可视化——通过自然语言描述生成 `ggplot2` 代码（如 ChatGPT 的"生成一张按年份分组的箱线图"）。你在这里学到的图形语法思想，同样适用于 Python 的 `plotnine`（`ggplot2` 的 Python 移植）和 Vega-Lite（声明式可视化的 JSON 规范）。

## 第五部分 部署与可重复性

### 第 13 章 模型部署与 API 服务

核心库：FastAPI, uvicorn, pydantic

#### 13.1 你能学到什么

本章教你将训练好的模型部署为可被他人调用的 Web 服务。你将学会：使用 FastAPI 构建 RESTful API（当前 Python 最快的 Web 框架之一）；定义输入输出的数据模式（Pydantic 模型）；使用 *uvicorn* 启动生产级服务器（存疑）；编写 API 文档（FastAPI 自动生成 Swagger UI）。

#### 13.2 为什么需要部署

训练好的模型如果只存在于你的笔记本中，它只是一堆数字。只有部署为 API（应用程序编程接口），其他人才能在不安装 Python 环境的情况下使用你的模型：通过浏览器或一个简单的 HTTP 请求，输入环境条件，获得管理建议和产量预测。部署也让你的工作可以被引用和复现。

#### 13.3 核心代码

```
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
import numpy as np
```

```
app = FastAPI(title="2026-SP Management Optimizer")
```

```
# 定义输入数据模式
```

```
class EnvironmentalInput(BaseModel):
```

```
    bio1: float # 年均温
```

```
    bio12: float # 年降水
```

```
    sand: float # 砂粒含量
```

```
    soc: float # 土壤有机碳
```

```
    elev: float # 海拔
```

```
    # ... 其余 60 个特征字段
```

```
# 定义输出数据模式
```

```
class ManagementOutput(BaseModel):
```

```
    n_optimal: float # 最优施氮量 (kg/ha)
```

```
    irr_optimal: float # 最优灌溉量 (mm)
```

```
    yield_predicted: float # 预测产量 (t/ha)
```

```
    yield_ci_lower: float # 产量 95%CI 下界
```

```
    yield_ci_upper: float # 产量 95%CI 上界
```

```
# 启动时加载模型
```

```
@app.on_event("startup")
```

```
async def load_models():
```

```
    global model, scaler
```

```
    model = joblib.load("model_xgb.pkl")
```

```
    scaler = joblib.load("scaler.pkl")
```

```
@app.post("/predict", response_model=ManagementOutput)
```

```
async def predict(input_data: EnvironmentalInput):
```

```
    features = np.array([[input_data.bio1, input_data.bio12, ...]])
```



```
features_scaled = scaler.transform(features)
pred = model.predict(features_scaled)[0]
return ManagementOutput(
    n_optimal=pred[0], irr_optimal=pred[1],
    yield_predicted=pred[2],
    yield_ci_lower=pred[3], yield_ci_upper=pred[4]
)
```

启动服务：在命令行运行 `uvicorn app:app --host 0.0.0.0 --port 8000`，然后打开浏览器访问 `http://localhost:8000/docs`，即可看到自动生成的交互式 API 文档（Swagger UI），可以直接在网页上测试 API。

## 13.4 掌握后的延展能力

API 部署是所有 AI 产品从"实验室模型"走向"实际应用"的关键一步。掌握 FastAPI 后，你可以：将任何 Python 函数包装为可供 Web/App 调用的微服务；参与云原生架构的设计与开发；理解后端开发的基本范式（请求-响应模型、中间件、异步处理）。当前趋势（2025-2026）是"MLOps"——机器学习运维，将模型训练-部署-监控形成闭环。你在这里学到的 API 部署是 MLOps 链路中"部署"环节的核心技能。

## 第 14 章 容器化与可重复研究

核心工具：Docker

### 14.1 你能学到什么

本章教你使用 **Docker** 打包整个项目的运行环境。你将学会：编写 Dockerfile 定义项目的完整环境；构建 Docker 镜像并推送到 Docker Hub；使用 docker-compose 管理多服务编排；理解容器化如何确保研究的可重复性。

### 14.2 "在我的机器上能跑"的问题

这是科学计算最常见的困境：你的代码在你的 Windows 笔记本电脑上完美运行，但 coworker 的 Mac 上报错"找不到 geos\_c.dll"；审查人的 Linux 服务器上报错"GLIBC 版本不匹配"；你自己的电脑重装系统后环境全部丢失。**Docker** 通过将应用程序及其所有依赖（包括操作系统级库）打包为一个轻量级的"容器"，确保在任何机器上运行结果完全一致。

### 14.3 项目 Dockerfile

```
FROM python:3.11-slim

# 安装系统级依赖（GDAL、proj 等地理空间库）

RUN apt-get update && apt-get install -y \
    gdal-bin libgdal-dev libproj-dev && \
    rm -rf /var/lib/apt/lists/*
```

```
# 设置工作目录
WORKDIR /app

# 复制依赖文件并安装
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# 复制源代码
COPY src/ ./src/
COPY config.yaml .

# 暴露 API 端口
EXPOSE 8000

# 启动命令
CMD ["uvicorn", "src.api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

## 14.4 可重复研究的最佳实践

除了 Docker 容器化，可重复研究还需要以下配套措施：

**随机种子固定：**在所有涉及随机性的操作中设置固定种子（`numpy.random.seed(42)`, `xgboost` 的 `random_state` 参数, `sklearn` 的 `random_state`）。这是可重复性的最基本要求。

**数据版本化：**使用 `DVC`（`Data Version Control`）或直接下载日期标注原始数据，确保所有合作者使用完全相同的数据版本。

**配置外部化：**所有可调参数（文件路径、模型超参数、网格分辨率）抽取到 `config.yaml` 文件中，禁止硬编码在 Python 代码中。修改参数只需编辑配置文件，无需改动代码。

**日志记录：**使用 Python 的 `logging` 模块（而非 `print`）记录训练过程，包括时间戳、损失值、验证分数。这对于后续排查"为什么上次训练效果好而这次不好"至关重要。

## 14.5 掌握后的延展能力

容器化是现代软件工程和云原生架构的基础。掌握 Docker 后，你可以：将任何应用部署到 Kubernetes 集群进行自动扩缩容；使用 GitHub Actions 自动构建和推送镜像（CI/CD）；参与微服务架构的设计。当前趋势（2025-2026）是"WebAssembly"（Wasm）作为轻量级容器替代方案的兴起——它比 Docker 启动更快（毫秒级），安全性更高，但在生态成熟度上仍落后于 Docker。

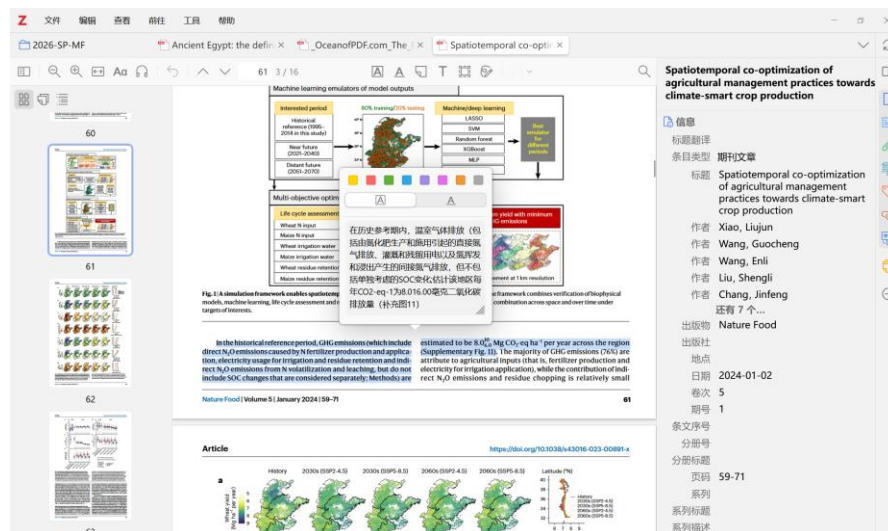
## 第 15 章 学术写作与文献管理

### 15.1 你能学到什么

本章介绍学术论文写作和文献管理的工具与实践。你将学会：使用 Zotero 管理参考文献并自动生成格式化引用；遵循学术写作的结构规范（IMRaD 格式）；使用 Markdown 或 LaTeX 撰写论文初稿；在写作过程中保持"去 AI 化"——使文字具有个人风格而非明显的 AI 生成痕迹。

### 15.2 文献管理推荐：Zotero

Zotero 是免费开源的文献管理工具，推荐原因：浏览器插件一键抓取论文元数据（标题、作者、期刊、DOI）；自动重命名和整理 PDF 文件；Word/LibreOffice 插件支持"引用-刷新"即时生成参考文献列表；支持共享文献库（团队协作时所有人使用同一套参考文献）；支持多样插件使用（如翻译插件与笔记插件）。



15.3 论文结构设计（如需撰写）

推荐使用 IMRaD 结构（Introduction, Methods, Results, and Discussion），这是绝大多数 SCI 期刊的标准格式。以下为本项目的具体章节设计：

| 章节                    | 内容                                                                                                                | 预计字数    |
|-----------------------|-------------------------------------------------------------------------------------------------------------------|---------|
| Introduction          | 研究背景、华北平原小麦-玉米生产的环境挑战、前人工作 (Xiao et al. 2024)、本研究的目标与创新点                                                          | ~800 词  |
| Materials and Methods | 2.1 研究区域(平谷区); 2.2 数据来源(逐源列表); 2.3 环境指纹构建方法; 2.4 XGBoost 模型训练与验证; 2.5 NSGA-II 优化; 2.6 SHAP 解释; 2.7 Bootstrap 不确定性 | ~1500 词 |
| Results               | 3.1 模型性能; 3.2 优化管理方案 vs 农户实践; 3.3 SHAP 特征重要性分析; 3.4 育种方向预测; 3.5 不确定性分析                                            | ~1200 词 |
| Discussion            | 4.1 管理优化方案的可操作性; 4.2 1km 分辨率的有效性; 4.3 品种数据的局限性; 4.4 与 Xiao et al.的对比; 4.5 未来方向                                    | ~1000 词 |
| Conclusion            | 简明总结主要发现和意义                                                                                                       | ~200 词  |

15.4 写作规范与去 AI 化

学术论文写作中需特别注意以下规范：

- 1）全文使用被动语态为主（"The model was trained"而非"We trained the model"），部分期刊允许第一人称但需保持一致；
- 2）数据和单位之间留空格（"180 kg/ha"而非"180kg/ha"）；
- 3）所有缩写首次出现时给出全称（"Growing Degree Days (GDD)"）；

4) 参考文献格式严格遵循目标期刊的要求（如 *Journal of Integrative Agriculture* 使用 APA 格式）。

**关于去 AI 化：**AI 辅助写作是合理的，但直接使用 AI 生成且未经修改的文字容易被识别。以下方法可以降低 AI 痕迹：将长句拆分为短句（AI 喜欢嵌套从句）；使用你自己熟悉的表达方式替换 AI 的套话（如将 "In the context of" 替换为具体的情境描述）；主动使用领域内的术语而非 AI 喜欢的通用表达（如 "粒重" 而非 "籽粒重量参数"）；在 Discussion 中引用你自己阅读过的文献的具体结论，而非 AI 生成的泛泛讨论。

## 15.5 掌握后的延展能力

学术写作是所有研究者的基本功。掌握了 IMRaD 结构和文献管理后，你可以高效地撰写任何学科的学术论文、学位论文和基金申请书。当前趋势（2025-2026）是 "动态文档"（Dynamic Documents）——使用 Quarto 或 Jupyter Book 将代码、图表和文本整合为可自动更新的出版级文档。掌握这一技能意味着你的论文图表可以随着数据更新而自动刷新，无需手动重新截图替换。

# 附录

## 附录 A 推荐学习路径（3 周速成方案）

| 时间    | 学习内容                                      | 产出                                |
|-------|-------------------------------------------|-----------------------------------|
| 第 1 周 | Git 基础 + Python 环境搭建 + pandas 核心操作        | 能独立 clone 仓库、创建分支、用 pandas 处理表格数据 |
| 第 2 周 | rasterio/geopandas + XGBoost 基础 + SHAP 初步 | 能提取栅格数据、训练简单的 XGBoost 模型并解释特征重要性  |
| 第 3 周 | NSGA-II/遗传算法 + R 语言 ggplot2 + FastAPI 基础  | 能运行多目标优化、制作出版级图表、部署简单的 API        |

## 附录 B 常见问题排查

### Q: rasterio 导入报错"DLL load failed"

A: 原因：Windows 缺少 GDAL 的 DLL 依赖。解决：使用 conda 安装而非 pip（conda install -c conda-forge rasterio），conda 会自动处理系统级依赖。

### Q: XGBoost 训练报错"Check failed: valid"

A: 原因：数据中包含 NaN 或无穷值。解决：训练前检查 df.isnull().sum()和 np.isinf(X).sum()。

### Q: Git push 被拒绝"Updates were rejected"

A: 原因：远程分支有新的提交，你本地的分支落后了。解决：git pull --rebase origin main，解决冲突后再 push。



### **Q: NSGA-II 收敛慢或未收敛**

A: 原因：决策变量范围过大或种群数量太小。解决：增大 pop\_size 到 200+，增加 n\_gen 到 500+，确保每个决策变量的上下界合理。

### **Q: ggplot2 中文字体显示为方块**

A: 原因：R 未配置中文字体。解决：安装 showtext 包，使用 showtext\_auto() 启用，然后用 font\_add() 注册中文字体。

### **Q: SHAP summary\_plot 报错 "matplotlib error"**

A: 原因：SHAP 默认绘图与某些 matplotlib 版本不兼容。解决：升级到 shap>=0.42 和 matplotlib>=3.7，或使用 shap.plots.beeswarm 代替。

# 附录 C 术语对照表

| 英文术语                | 中文           | 释义                               |
|---------------------|--------------|----------------------------------|
| XGBoost             | 极限梯度提升       | 基于梯度提升的集成学习算法，结构化数据建模的首选         |
| SHAP                | Shapley 加法解释 | 基于博弈论 Shapley 值的模型解释方法           |
| NSGA-II             | 非支配排序遗传算法 II | 基于帕累托最优的多目标进化优化算法                |
| GDD                 | 生长期日         | 作物发育所需的热量累积指标，单位为度 C-日           |
| Bootstrap           | 自助法          | 通过有放回重采样估计统计量分布的方法               |
| Raster              | 栅格数据         | 由规则网格组成的空间数据，如 GeoTIFF 卫星影像      |
| CRS                 | 坐标参考系        | 定义地理坐标与平面坐标转换关系的系统               |
| Affine Transform    | 仿射变换         | 描述栅格像素行列号与地理坐标之间的线性变换            |
| Feature Engineering | 特征工程         | 将原始数据转化为模型可用的特征向量的过程             |
| Overfitting         | 过拟合          | 模型在训练集上表现极好但在新数据上表现差的现象          |
| Cross Validation    | 交叉验证         | 将数据分为 K 份，轮流用 K-1 份训练 1 份验证的评估方法 |
| Hyperparameter      | 超参数          | 在训练前设定的参数（如树深度、学习率），不能从数据中学习     |
| Pareto Front        | 帕累托前沿        | 多目标优化中所有非支配解的集合                  |
| Surrogate Model     | 代理模型         | 用于替代复杂昂贵模拟的快速近似模型                |
| CI/CD               | 持续集成/持续部署    | 自动测试和部署代码的 DevOps 实践             |
| API                 | 应用程序编程接口     | 软件组件之间通信的标准化接口                   |
| Docker              | 容器化平台        | 将应用及其依赖环境打包为可移植容器的工具             |
| MLOps               | 机器学习运维       | 将机器学习模型从开发到部署的完整生命周期管理           |

---

本手册版本 1.0 | 2026 年 7 月 | 如有更新将在 **GitHub** 仓库中同步发布。

项目仓库：[待创建]

问题反馈：请在 **GitHub Issues** 中提交。